

VR-Robo: A Real-to-Sim-to-Real Framework for Visual Robot Navigation and Locomotion

Shaoting Zhu^{12*}, Linzhan Mou^{23*}, Derun Li²⁴, Baijun Ye¹³, Runhan Huang¹², Hang Zhao^{†123}

*Equal contribution †Corresponding author [VR-Robo.github.io](https://github.com/VR-Robo)

Abstract

Recent success in legged robot locomotion is attributed to the integration of reinforcement learning and physical simulators. However, these policies often encounter challenges when deployed in real-world environments due to sim-to-real gaps, as simulators typically fail to replicate visual realism and complex real-world geometry. Moreover, the lack of realistic visual rendering limits the ability of these policies to support high-level tasks requiring RGB-based perception like ego-centric navigation. This paper presents a *Real-to-Sim-to-Real* framework that generates photorealistic and physically interactive “digital twin” simulation environments for visual navigation and locomotion learning. Our approach leverages 3D Gaussian Splatting (3DGS) based scene reconstruction from multi-view images and integrates these environments into simulations that support ego-centric visual perception and mesh-based physical interactions. To demonstrate its effectiveness, we train a reinforcement learning policy within the simulator to perform a visual goal-tracking task. Extensive experiments show that our framework achieves RGB-only sim-to-real policy transfer. Additionally, our framework facilitates the rapid adaptation of robot policies with effective exploration capability in complex new environments, highlighting its potential for applications in households and factories.

1. Introduction

Exploring, perceiving, and interacting with the physical real world is crucial for legged robotics, which supports many applications such as household service and industrial automation. However, conducting real-world experiments poses considerable challenges due to safety risks and efficiency constraints. Consequently, training in simulation[21, 26, 30] has emerged as an effective alternative, allowing robots to experience diverse environmental conditions, safely explore failure cases, and learn directly from their actions. Despite these advantages, transferring policies learned in simulated environments to real world remains challenging, owing to the significant *Sim-to-Real* reality

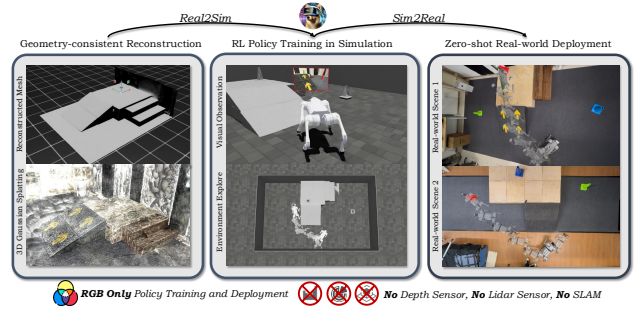


Figure 1. **Our VR-Robo introduces a unified real-to-sim-to-real framework.** We generate photorealistic and physically interactive “digital twin” simulation environments for visual policy training, and enable zero-shot real-world deployment.

gap[28, 30].

Substantial efforts have been made to narrow the *Sim-to-Real* gap in legged locomotion. Previous studies have employed cross-domain depth images to train agents in simulation, achieving impressive zero-shot policy transfer for both quadruped[6, 12, 34, 41] and humanoid[42] robots. To further integrate RGB perception into the *Sim-to-Real* pipeline, LucidSim[38] leverages generative models within simulation[30] for visual parkour. However, these depth-based and generation-based visual policies are predominantly constrained to low-level locomotion tasks, primarily because standard simulators struggle to replicate the visual fidelity and complex geometry of real-world environments.

Recently, advanced neural reconstruction techniques such as Neural Radiance Fields (NeRF)[22] and 3D Gaussian Splatting (3DGS)[16] have emerged as promising solutions for creating *Real-to-Sim* “digital twins” from real-world data. Nonetheless, most existing approaches[2, 18, 19, 35, 36] focus on enhancing photorealism, offering limited support for physical interaction with complex terrains. Moreover, the simulations in these works often lack mechanisms for robust environment exploration, thus constraining their deployment in complex real-world scenarios that demand dynamic interaction.

To address these challenges, we introduce VR-Robo, a novel *Real-to-Sim-to-Real* framework that enables realistic, interactive *Real-to-Sim* simulation and reinforcement

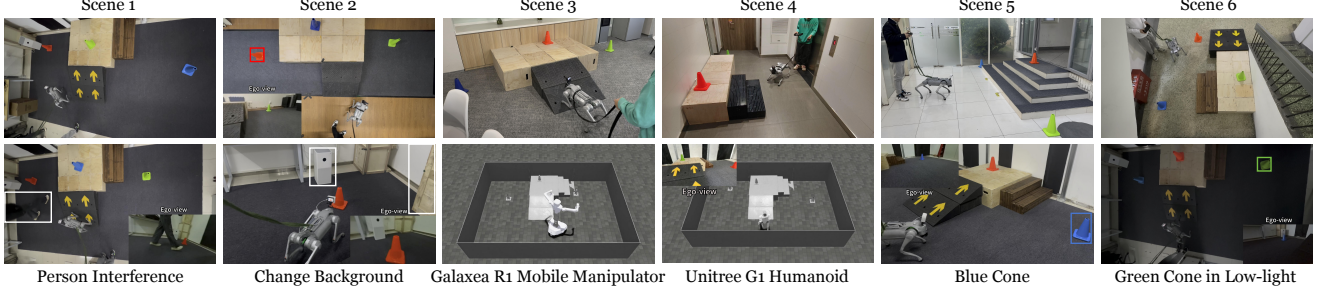


Figure 2. Diverse experiment settings including scenes, conditions, and robot types (better viewed when zoomed in).

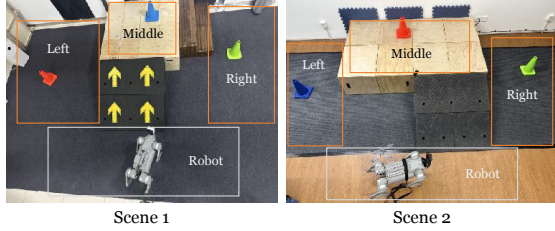


Figure 3. **Task definition.** The robot aims to reach the target cone of a specified color in a specified spatial location, relying solely on *ego-view* RGB observation and proprioception.

learning (RL) policy training for legged robot navigation and locomotion. Given the multi-view images, we employ 3DGS[16] and foundation-model priors[37] to reconstruct geometry-consistent scenes and objects. We then propose a GS-mesh hybrid representation with coordinate alignment to create a “digital twin” simulation environment in Isaac Sim, which supports ego-centric visual perception and mesh-based physical interactions. To enable robust policy training, we introduce an agent-object randomization and occlusion-aware scene composition strategy, as illustrated in Figure 7.

We summarize our contributions as follows:

1. **A *Real-to-Sim-to-Real* framework for robot navigation and locomotion.** We propose to reduce the *Sim-to-Real* gap by training RL policies within a realistic and interactive simulation environment reconstructed only from RGB images captured from real-world.
2. **Photorealistic and physically interactive *Real-to-Sim* environment reconstruction.** We introduce a novel pipeline for transferring real-world environments into simulation using a GS-mesh hybrid representation with coordinate alignment. We further incorporate object randomization and occlusion-aware scene composition for robust and efficient RL policy training.
3. **Zero-shot *Sim-to-Real* RL policy transfer.** Through extensive experimental evaluations, we demonstrate that VR-Robo produces effective navigation and locomotion policies in complex, real-world scenarios using RGB-only observations. In addition, we have some empirical findings for RL policy training, including the gen-

eralization ability, sample complexity, and architectural choices.

2. Related Works

2.1. Sim-to-Real Policy Transfer

Transferring reinforcement learning (RL) policies trained in simulation to the real world remains a major challenge due to substantial domain gaps. Traditional simulator-based methods, such as domain randomization[23, 29, 33] and system identification[15, 27], aim to reduce the *Sim-to-Real* gap by aligning simulations with physical setups. Recent work[5, 17] proposes leveraging conditional generative models to augment visual observations for more robust agent training. LucidSim[38], for example, incorporates RGB color perception into the *Sim-to-Real* pipeline to learn low-level visual parkour by generating and augmenting image background. However, these approaches remain constrained by conventional simulators, which fail to capture the full breadth of real-world physics and visual realism necessary for high-level policy training and real-world deployment.

2.2. Real-to-Sim Scene Transfer

Recently, advances in scene representation and reconstruction such as Neural Radiance Fields (NeRF)[22] and 3D Gaussian Splatting (3DGS)[16] have facilitated the creation of high-fidelity digital twins that closely replicate real-world environments for *Real-to-Sim* scene transfer. For instance, NeRF2Real[2] integrates NeRF and mesh into a simulation for vision-based bipedal locomotion policy training, but struggles to achieve real-time rendering, terrain climbing, and environment exploration, thus limiting their practical application. Meanwhile, RialTo[31] augments digital twins with articulated USD representations to enable manipulative interactions. More recent works employ 3DGS to generate realistic simulations for both robot manipulation[18, 19, 35] and navigation[24, 36]. In contrast, VR-Robo is designed to produce photorealistic and physically interactive “digital twin” environments specifically tailored for ego-centric visual locomotion learning.

3. Task definition

Our designed task requires both high-level understanding and navigation, as well as low-level motion control. As shown in Figure 3, the task is defined as reaching the target cone of a specified color within a limited time horizon. Each scene features a terrain in the center composed of a high table and a slope or stair connecting the ground to the platform.

At the start of each trial, the robot’s initial position is sampled from $\mathbf{p}_{\text{robot}} \sim \mathcal{U}(\mathbb{P}_{\text{robot}})$, and its orientation is similarly randomized as $\theta_{\text{robot}} \sim \mathcal{U}(\theta_{\text{robot}})$. Each scene contains three cones with identical shapes and textures but different colors $\mathcal{C} = \{\text{red, green, blue}\}$. These cones are placed in three designated regions: left, middle, and right, with one cone per region at a random position $\mathbf{p}_{\text{cone}} \sim \mathcal{U}(\mathbb{P}_{\text{cone}}^{\text{region}})$. Since the robot may not initially face the target cone, it must explore the environment to identify and locate the correct cone. Additionally, if the target cone is placed atop the table, the robot must navigate up the slope to access the platform.

4. A Real-to-Sim-to-Real System for Visual Locomotion

4.1. Geometry-Consistent Reconstruction

Given multi-view RGB images of a scene with corresponding camera poses from COLMAP[25], we aim to generate a photorealistic and physically interactive simulation environment for agent policy training. We decoupledly reconstruct the environment and the object, and then compose them into a single Gaussian scene for policy training.

Gaussian-based 3D Scene Representation. 3D Gaussian Splatting (3DGS) represents the scene as a set of Gaussian primitives as follows:

$$\mathcal{G}_i(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right), \quad (1)$$

During optimization, $\boldsymbol{\Sigma}_i$ is reparametrized using a scaling matrix $S_i \in \mathbb{R}^3$ and a rotation matrix $R_i \in \mathbb{R}^{3 \times 3}$ as $\boldsymbol{\Sigma}_i = R_i S_i S_i^T R_i^T$. The color value of a pixel $\mathbf{c}(x)$ can be rendered through a volumetric alpha-blending[16] process:

$$\mathbf{c}(x) = \sum_{i \in N} T_i c_i \alpha_i(\mathbf{x}), \quad T_i = \prod_{i=1}^{i-1} (1 - \alpha_i(\mathbf{x})). \quad (2)$$

where $\alpha_i(\mathbf{x}) = o_i \mathcal{G}_i(\mathbf{x})$ denotes the alpha of the Gaussian $\mathcal{G}_i$ at $\mathbf{x} \in \mathbb{R}^3$. Meanwhile, the view-dependent color c_i of $\mathcal{G}_i$ is represented by its spherical harmonics (SH) coefficients.

Flattening 3D Gaussians for Geometric Modeling. Inspired by[14], we flatten the 3D Gaussian ellipsoid with covariance $\boldsymbol{\Sigma}_i = R_i S_i S_i^T R_i^T$ into 2D flat planes to enhance the scene geometry modeling by minimizing its shortest axis scale $S_i = \text{diag}(s_1, s_2, s_3)$:

$$\mathcal{L}_{\text{scale}} = \frac{1}{N} \sum_{i \in N} \|\min(s_1, s_2, s_3)\|, \quad (3)$$

where N denotes the total number of planar-based Gaussian splats. Following PGSR[4], we utilize the plane representation to render both the plane-to-camera distance map $\mathcal{D}$ and the normal map $\mathcal{N}$, then convert them into unbiased depth maps by intersecting rays with the corresponding planes as:

$$D(p)_{\text{render}} = \frac{\mathcal{D}}{\mathcal{N}(p)K^{-1}\tilde{p}}. \quad (4)$$

where p is the 2D position on the image plane. $\tilde{p}$ denotes the homogeneous coordinate of p , and K is the camera intrinsic.

Foundation Model Geometric Prior Constraints. In texture-less regions (e.g., ground and walls), photometric loss tend to be insufficient. We employ an off-the-shelf monocular depth estimator[37] to provide dense depth prior. We address the inherent scale ambiguity between the estimated depths and the actual scene geometry by comparing them to sparse Structure-from-Motion (SfM) points, following[32]. Specifically, we align the scale of the monocular depth map D_{mono} with the SfM points projected depth map D_{sfm} using linear regression:

$$\hat{s}, \hat{t} = \arg \min_{s, t} \sum_{p \in D_{\text{sfm}}} \|s * D(p)_{\text{mono}} + t - D(p)_{\text{sfm}}\|_2^2. \quad (5)$$

Finally, we use the re-scaled depth map $\hat{s} \times D(p)_{\text{mono}} + \hat{t}$ to regularize the rendered depth map $D(p)_{\text{render}}$. Similarly, we adopt an off-the-shelf monocular normal estimator[1] to provide dense normal regularization to the rendered normal map $\mathcal{N}_{\text{render}}$ for accurate geometric modeling.

Multi-view Consistency Constraints. Inspired by[9, 10], a patch-based normalized cross-correlation (NCC) loss is applied between two gray renders $\{\mathbf{I}_{\text{render}}, \hat{\mathbf{I}}_{\text{render}}\}$ to force the multi-view photometric consistency:

$$\mathcal{L}_{\text{mv}} = \frac{1}{\|\mathcal{P}\|} \sum_{p \in \mathcal{P}} \sum_{p \in \mathcal{P}} (1 - \text{NCC}(\hat{\mathbf{I}}(\mathcal{H}p), \mathbf{I}(p))). \quad (6)$$

where $\mathcal{P}$ is the set of all patches extracted from $\mathbf{I}_{\text{render}}$ and $\mathcal{H}$ is the homography matrix between the two frames.

4.2. Building Realistic and Interactive Simulation

To enable the agent-environment interaction, we integrate a GS-mesh hybrid scene representation into the Isaac Sim with coordinate alignment. We further leverage agent-object randomization and occlusion-aware scene composition to advance robust and generalizable visual policy training.

GS-mesh Hybrid Representation. 1) The *Gaussian* representation generates photorealistic visual observations from the robot’s ego-centric viewpoints. We first align the intrinsic parameters between *Sim* and *Real* by calibrating the

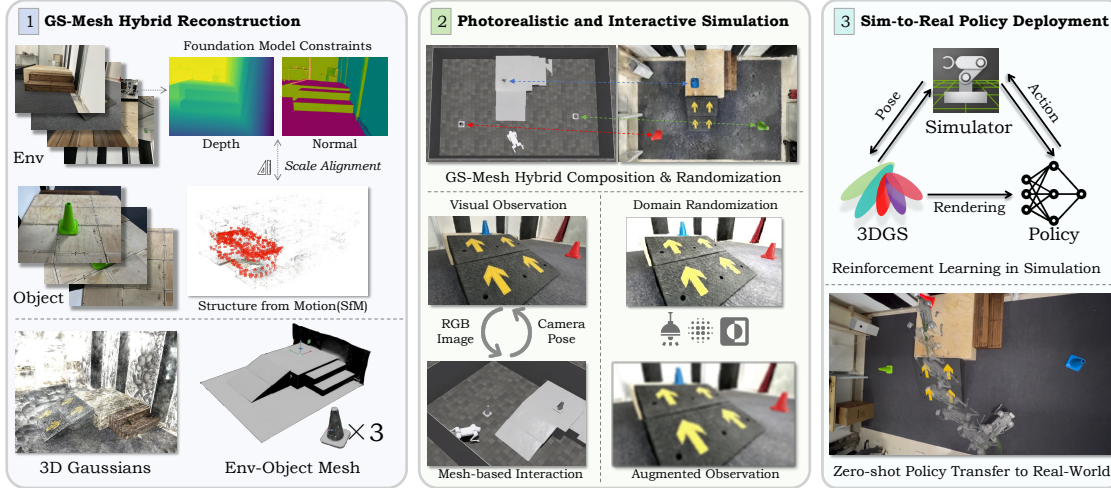


Figure 4. **VR-Robo real-to-sim-to-real framework.** We build a realistic and interactive simulation environment with GS-mesh hybrid representation and occlusion-aware composition & randomization for policy training. Finally, we zero-shot transfer the RL policy trained in simulation into the real robot for ego-centric navigation and visual locomotion.

robot camera’s focal length and distortion parameters. We then obtain the camera extrinsic within the simulation coordinate system by retrieving the ego-view position and quaternion-based orientation from Isaac Sim. 2) The **Mesh** representation facilitates physical interaction and precise collision detection. We render the depth map for each input view and leverage a Truncated Signed Distance Function (TSDF)[7] fusion algorithm to build the corresponding TSDF field. We then extract the mesh from this TSDF field to enable physically accurate interactions within the simulation.

Coordinate Alignment. To align the reconstructed COLMAP coordinate system with the Isaac Sim environment, we first compute the homogeneous transformation matrix $T_{\text{homo}} \in \mathbb{R}^{4 \times 4}$ by manually matching four non-coplanar points.

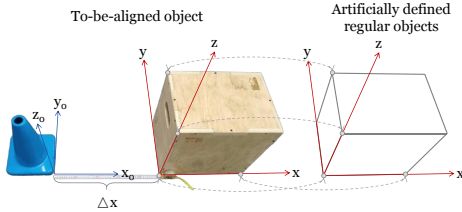


Figure 5. **Coordinate alignment.** We first compute the transformation for a regular reference object and then apply this transformation to the irregular object of interest.

Specifically, as shown in Figure 5, we use objects with regular shapes to transform coordinate systems. For instance, to obtain the transformation matrix for the blue cone, we first manually define a rectangular block $\mathcal{B}$ with the correct coordinate system and size as the reference object. Then, using the four-point registration method, we align the block’s coordinates with the reconstructed

COLMAP system. The transformation matrix T_{block} is determined as:

$$T_{\text{block}} = \arg \min_T \sum_{i=1}^4 \|T \cdot \mathbf{p}_i^{\mathcal{B}} - \mathbf{p}_i^{\text{COLMAP}}\|^2, \quad (7)$$

Finally, the transformation matrix for the cone, T_{cone} , is determined by translating the cone’s position relative to the aligned rectangular block:

$$T_{\text{cone}} = T_{\text{block}} \cdot \Delta \mathbf{p}_{\text{cone}}. \quad (8)$$

Gaussian Attributes Adjustment. We first decompose T_{homo} into its rotation component $R_{\text{homo}} \in \mathbb{R}^{3 \times 3}$, translation vector $t_{\text{homo}} \in \mathbb{R}^3$, and scale factor $s_{\text{homo}} \in \mathbb{R}$. In addition, the 3D covariance matrix Σ of the Gaussian points is parameterized by a scaling matrix $S \in \mathbb{R}^3$ and a rotation matrix $R \in \mathbb{R}^{3 \times 3}$, such that $\Sigma = RSS^T R^T$. The mean μ , scaling S and rotation R are adjusted as follows:

$$\mu' = R_{\text{homo}} \mu + t_{\text{homo}}, \quad (9)$$

$$S' = S + \log(s_{\text{homo}}), \quad (10)$$

$$R_{\text{norm}} = \frac{R_{\text{homo}}}{s_{\text{homo}}}, \quad \Sigma' = R_{\text{norm}} \Sigma R_{\text{norm}}^T, \quad (11)$$

Since the spherical harmonics (SHs) for the 3D Gaussians are stored in world space (i.e., the COLMAP coordinate system), view-dependent colors change when the Gaussians rotate. To accommodate different rotations, we first extract the Euler angles α, β, γ from the rotation matrix R_{homo} and construct the corresponding Wigner D-matrix D . We then apply D to rotate the SH coefficients. Formally, each band ℓ of the SH coefficients (of length $2\ell + 1$) transforms as:

$$\mathbf{C}^{(\ell)'} = D_{\ell}(\alpha, -\beta, \gamma) \mathbf{C}^{(\ell)}. \quad (12)$$

where $\mathbf{C}^{(\ell)}$ is the vector of the spherical-harmonic coefficients for degree ℓ and D_{ℓ} is the $(2\ell + 1) \times (2\ell + 1)$ Wigner D-matrix.

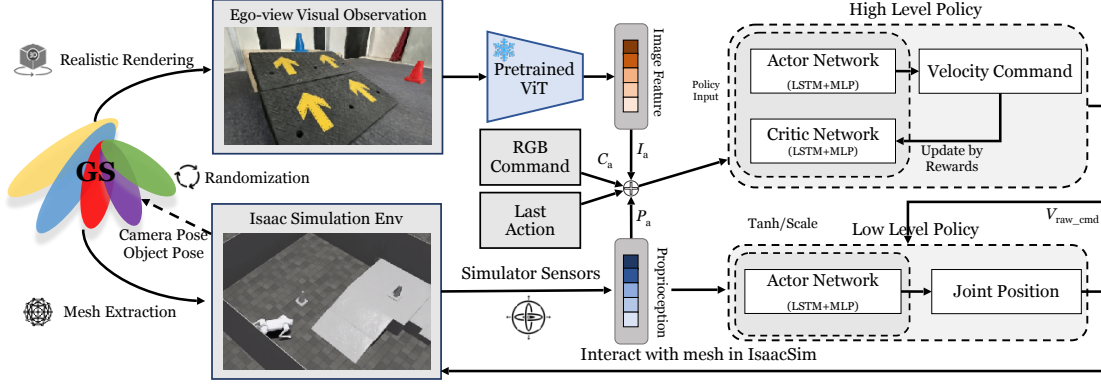


Figure 6. **RL policy training in the reconstructed simulation environment.** The agent leverages the ego-view GS photorealistic rendering as visual observations and interacts with the GS-extracted mesh in the Isaac Sim environment.

Occlusion-Aware Composition and Randomization.

With the resulting mesh and 3D Gaussians, we employ an interactive mesh editor to obtain both the 3D bounding box and its corresponding transformation matrix $T_{\text{bbox}} \in \mathbb{R}^{4 \times 4}$. Since the object mesh and the 3D Gaussians share the same COLMAP coordinate system, we use the mesh bounding box to crop the object Gaussians and merge them into the environment coordinate space according to the transformation below:

$$T_{\text{obj}}^{\text{COLMAP-env}} = T_{\text{sim}}^{\text{COLMAP-env}} \cdot T_{\text{COLMAP-obj}}^{\text{sim}} \cdot T_{\text{bbox}}. \quad (13)$$

As shown in Figure 7, the merged object Gaussians are synchronized with the object mesh in Isaac and can be rendered jointly with the environment Gaussians via a volumetric alpha-blending [16] process. Since they are separately reconstructed, there are no holes in the merged 3DGS scenes. By incorporating z-buffering [3], it is impossible to see those objects that are farthest away from the ego-viewer and behind other objects. Therefore, our method can achieve occlusion-aware scene composition with accurate visibility. This randomization strategy substantially increases the environment’s diversity, enabling more robust and scalable agent training.

4.3. Reinforcement Learning in Reconstructed Simulation

The next stage of VR-Robo focuses on training a robust locomotion policy via reinforcement learning, as illustrated in Figure 6. This policy is designed to address diverse goal-tracking tasks under varying environmental conditions. Due to the limited onboard computational resources of real robots, we adopt a two-level asynchronous control strategy: the high-level policy operates at 5 Hz, while the low-level policy executes at 50 Hz. The high-level policy communicates with the low-level policy via velocity commands, represented as $V_{\text{cmd}} = (V_x, V_y, V_{\text{yaw}})$. Main experiments are conducted and demonstrated on the Unitree Go2 quadruped robot.

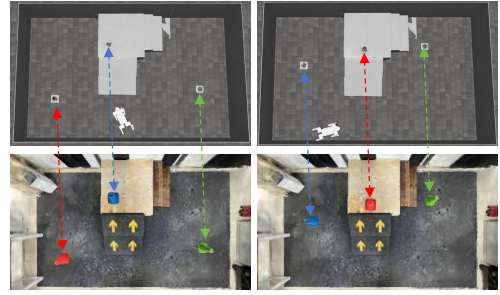


Figure 7. **Agent-object Randomization and Scene Composition.** At the beginning of each episode, we randomly sample the positions for the robot and three cones in the Isaac Sim environment (upper row). We synchronously merge the agent and object Gaussians into the environment and compose for joint rendering (lower row).

Low-Level Policy. The first step is to train the low-level policy, similar to previous locomotion works[34, 39]. The low-level policy takes velocity commands $V_{\text{cmd}} = (V_x, V_y, V_{\text{yaw}})$ along with robot proprioception as input, and outputs the desired joint positions. The actor network is composed of a lightweight LSTM layer combined with multiple MLP layers.

The low-level policy adopted here enables the quadruped robot to climb slopes and stairs measuring up to 15 cm in height. However, unlike previous parkour works[6, 12, 41], this policy does not allow the robot to directly climb onto terrain taller than 30 cm. While existing frameworks could be extended to handle higher climbs, our approach instead focuses on teaching the robot to recognize and utilize slopes or stairs to reach high platforms. This strategy is particularly useful in real-world scenarios featuring taller obstacles, where direct climbing or jumping may be infeasible for any policy.

High-Level Policy. We freeze the previously trained low-level policy and train the high-level policy independently. The high-level policy is trained using reinforcement learning (PPO) within our GS-mesh hybrid simulation environ-

ment.

State Space: The high-level policy actor network receives four types of inputs: 1) RGB Image Feature (I_a): We employ a frozen, pre-trained Vision Transformer (ViT)[8] to extract features from RGB images. ViT’s attention mechanism is well-suited for detecting positions and specific colors. This approach compresses the input size significantly and accelerates training. 2) RGB Command (C_a): A RGB vector indicating the desired color of the target cone. For example, $[1, 0, 0]$ represents the red cone. 3) Last Action: The action output by the policy during the previous timestep. 4) Proprioception (P_a): This includes the robot’s base angular velocity, projected gravity, joint positions, and joint velocities. We use an asymmetric actor-critic structure. The critic’s observation includes the actor’s observation (with noise removed) as well as the robot’s world coordinate position and orientation, and the target cone’s world coordinate position. This design improves the estimation of the value function and enhances the training process.

Action Space: The actor outputs the raw velocity command, $V_{\text{raw_cmd}} = (V_x, V_y, V_{\text{yaw}})$. This raw command will pass through a tanh layer and scaled by the velocity range $V_{\text{max_cmd}} = (V_{\text{max_x}}, V_{\text{max_y}}, V_{\text{max_yaw}})$ to ensure safety. Note that the tanh layer and velocity range scaling are applied outside the actor network. The resulting velocity command $V_{\text{cmd}} = (V_x, V_y, V_{\text{yaw}})$ is sent to the low-level policy, enabling the robot to move.

Reward Design: The total reward consists of two main categories: **Task Rewards** r_T and **Regularization Rewards** r_R . r_T include `Reach_goal`, `Goal_dis`, `Goal_dis_z`, and `Goal_heading`. To be specific, the robot receives a reward if it comes close enough to the goal.

$$r_{\text{reach_goal}} = \begin{cases} R_{\text{max}}, & \text{if } \|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{goal}}\|_2 \leq \epsilon, \\ 0, & \text{otherwise.} \end{cases} \quad (14)$$

`Goal_dis` is based on the change in the Euclidean distance to the goal between consecutive time steps:

$$r_{\text{goal_dis}} = d_{\text{prev}} - d_{\text{current}}, \quad (15)$$

$$d = \|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{goal}}\|_2. \quad (16)$$

`Goal_dis_z` is based on the change in the vertical (z-axis) distance to the goal:

$$r_{\text{goal_dis_z}} = d_{z,\text{prev}} - d_{z,\text{current}}, \quad (17)$$

$$d_z = |z_{\text{robot}} - z_{\text{goal}}|. \quad (18)$$

`Goal_heading`: This reward encourages the robot to face the goal by minimizing the yaw angle error, defined as a linear function of the yaw difference:

$$r_{\text{goal_heading}} = -|\psi_{\text{robot}} - \psi_{\text{goal}}|, \quad (19)$$

where ψ_{robot} is the robot’s current yaw angle, ψ_{goal} is the desired yaw angle toward the goal.

In addition, regularization rewards are incorporated to further refine the robot’s performance and ensure stable and efficient behavior. It contains `Stop_at_goal`, `Track_lin_vel`, `Track_ang_vel`, and `Action_l2`.

Training Process: We randomly sample the positions and orientations of the robot and cones at the beginning of each episode. The robot’s camera poses and cone poses are obtained from Isaac Sim and sent to the aligned and editable 3D Gaussians mentioned above to render a photorealistic image. The policy then outputs an action based on this image, which is applied in Isaac Sim to interact with the mesh.

Domain Randomization: We apply domain randomization to reduce the sim-to-real gap. This includes three key components: 1) **Camera Pose Noise:** Before requesting the 3D Gaussians to render an image, we add uniform noise to the camera pose extracted from Isaac Sim. 2) **Image Augmentation:** We randomly apply brightness, contrast, saturation, and hue adjustments to the image. This is followed by Gaussian blur to simulate camera blur caused by robot movement. Additionally, we randomly add Gaussian noise to the image with a small probability ($p = 0.05$). 3) **Image Delay:** We randomly apply a 0- or 1-step delay to the rendered images.

5. Experiments

Our experiments are designed to evaluate two main aspects of VR-Robo: **1)** its ability to reconstruct realistic and interactive simulation environments with GS-mesh hybrid representation, supporting the locomotion policy training through both visual observations and mesh interactions; **2)** its capability to minimize the *Sim-to-Real* gap when deployed to the real world. We conducted extensive experiments in both simulation and real-world settings to validate these capabilities.

5.1. Experiment Setup

Real-to-Sim Reconstruction. We reconstruct 6 distinct indoor room environments, each characterized by specific terrains. We also randomize the environments by incorporating three cones colored red, green, and blue. We use iPads or iPhones to take photos, which are easily available.

Simulated Locomotion Training. We conduct policy training in Isaac Sim using a single NVIDIA 4090D GPU. For the low-level policy, we instantiate 4,096 quadruped robot agents in parallel and train for 80,000 iterations from scratch. In contrast, the high-level policy is trained from scratch over 8,000 iterations with 64 quadruped agents. The entire training process requires approximately three days to complete. For visual encoding, we employ the Vision Transformer model “vit_tiny_patch16_224”, omitting its final classification head. The Isaac Sim simulator and the 3DGS rasterization-based renderer communicate via a TCP network.

Sim-to-Real Deployment. We deploy our policy on the Unitree Go2 quadruped robot, which is equipped with an NVIDIA Jetson Orin Nano (40 TOPS) as the onboard computer. We use ROS2[20] for communication between the high-level policy, low-level policy, and the robot. Both policies run onboard. The robot receives desired joint positions from the low-level policy for PD control ($K_p = 40.0$, $K_d = 1.0$). We use an Insta360 Ace camera to capture RGB images. The camera captures images at a resolution of 320×180 . After calibration, it has a horizontal field of view (FOVX) of 1.5701 radians and a vertical field of view (FOVY) of 1.0260 radians.

5.2. Simulation Experiments

We conduct comparison and ablation experiments with various baselines. For comparison experiments,

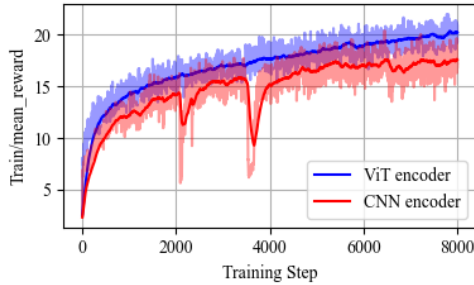


Figure 8. **Training reward comparison.** Pretrained CNN [13] encoder struggles to extract precise semantic features, leading to unstable and worse rewards compared to ViT [8] encoder.



Figure 9. Illustration of textured mesh and random background.

- **Imitation Learning:** We collect 60 different real-world trajectories via teleoperation and train the policy through supervised learning using regression optimization.
- **Random Background:** LucidSim [38] uses generative models to produce visual observations. Since the training and deployment code has not been open-sourced, we re-implemented it by using random images from ImageNet as the background and maintaining the target object.

For ablation experiments,

- **Textured Mesh:** We use SuGaR[11] to reconstruct textured mesh and support direct mesh rendering in Isaac Sim as visual observation.
- **CNN Encoder:** We replace ViT with a CNN [13] encoder, and remove the last classification layer.

We evaluated all methods in a ‘red cone reaching’ task within Scene 1. The red cone is randomly placed in one of three regions: left (ground), middle (terrain), or right (ground). We generate 150 distinct cone placements, specified before any experiments, and apply the same setup for all methods to ensure a fair comparison.

To measure performance, we report both Success Rate (SR) and Average Reaching Time (ART). An episode is deemed successful if the robot comes within 0.25 m of the red cone at any point within a 15-second window. For successful episodes, we record the time taken to reach the cone; otherwise, we assign the maximum time of 15 seconds. The ART is then computed as the mean reaching time across all trials.

The comparison and ablation results are shown in Table 2 and Table 3, respectively. Our method consistently achieves higher performance across all evaluation metrics by a large margin. In particular, the Imitation Learning (IL) baseline suffers from insufficient data samples and a lack of policy exploration. As shown in Figure 8, the CNN encoder struggles to extract precise features from the images. In this work, we reconstruct solely from RGB images. The textured mesh obtained from GS exhibits noticeable bulges in texture-less regions such as the ground and walls, as shown in Figure 9 and Table 4. These artifacts degrade the rendering quality and can even hinder the robot’s movement. The quality and resolution of the mesh itself significantly affect rendering fidelity, whereas GS is less sensitive to such factors. In contrast, our method crops and utilizes only the mesh of the central terrain, and renders observation from GS, effectively avoiding these issues. The Random Background setting (the bottom-right image) loses important features specific to the original scene (the bottom-left image). These scene-specific features are crucial for the robot to effectively explore the environment.

5.3. Real-World Experiments

We conduct qualitative experiments under varying conditions, including 6 different scenes, various target cone colors, changes in lighting conditions, random interference, background variations, and training on the different robots, as shown in Figure 2. A detailed demonstration is provided in the supplementary materials. These experiments highlight the robustness of our method, showcasing its adaptability to a wide range of environments and conditions.

We further conduct quantitative experiments, as presented in Table 1. In addition to the methods mentioned above, we conduct two additional baseline methods for real-world experiments. These methods are specifically designed in the real-world setting and cannot be directly compared in simulation:

- **SARO [40]:** SARO enables the robot to navigate across 3D terrains leveraging the vision-language model (VLM).

Table 1. Comparison and ablation experimental results in the real-world setting.

Method	Exteroception	Success Rate $\uparrow$			Average Reaching Time (s) $\downarrow$		
		Easy	Medium	Hard	Easy	Medium	Hard
Ours	RGB	100.00%	93.33%	100.00%	4.96	6.28	9.09
Imitation Learning (IL)	RGB	0.00%	0.00%	0.00%	15.00	15.00	15.00
SARO [40]	RGB	66.67%	26.67%	0.00%	46.49	57.24	60.00
Textured Mesh	RGB	20.00%	6.67%	0.00%	12.90	14.90	15.00
CNN Encoder	RGB	73.33%	66.67%	6.67%	9.10	11.41	14.90
w/o Domain Randomization	RGB	53.33%	6.67%	0.00%	10.04	14.76	15.00

Table 2. Comparison results in the simulation setting.

Method	SR $\uparrow$	ART (s) $\downarrow$
Ours	100.00%	4.94
Imitation Learning (IL)	8.67%	14.01
Random Background	43.33%	11.75

Table 3. Ablation results in the simulation setting.

Method	SR $\uparrow$	ART (s) $\downarrow$
Ours	100.00%	4.94
Textured Mesh	22.00%	12.73
CNN Encoder	54.67%	10.04

Table 4. Reconstruction quality comparison.

Method	Planar 3DGS	Textured Mesh
Ours	70.72	77.72
Textured Mesh	22.00%	12.73
CNN Encoder	54.67%	10.04

Figure 10. Tasks of different difficulties (*easy*, *medium*, and *hard*) and different target positions (*left*, *middle*, and *right*).

- **w/o Domain Randomization:** We exclude domain randomization as described in Section 4.3.

All the real-world quantitative experiments is in the ‘red cone reaching’ task setting in Scene 1. We categorize the task into three levels of difficulty based on the robot’s starting position: *easy*, *medium*, and *hard*. For *easy* tasks, the robot starts with the target cone directly visible. For *medium* tasks, the robot needs to turn to a certain degree to see the target cone. For *hard* tasks, the robot starts very far from the cone and faces a nearly opposite direction to the target cone. We randomly select 3 sets of cone positions, with each set containing one easy, one medium, and one hard task, as shown in Figure 10. For each task, we repeat the experiment 5 times. We then calculate the Success Rate (SR)

and Average Reaching Time (ART). For SARO, the maximum time is extended to 60 seconds due to the long inference time required by the Vision-Language Model (VLM). In the real robot experiments, we consider the task successful if the robot makes contact with the target cone.

Our method achieves the highest success rates across all difficulty levels and consistently records the shortest average reaching times, demonstrating its efficiency and robustness. Note that due to inherent randomness in the real-world setting, there is a failure case in the *medium* task, even though all trials succeed in the *hard* task. Among the other methods, SARO achieves moderate success on easy tasks (66.67%) but performs poorly on medium and hard tasks. This under-performance is primarily attributed to the lack of historical context and limited exploration capability in complex scenarios. If the robot cannot initially see the goal, it is unlikely to succeed.

6. Limitations and Conclusion

6.1. Limitations

The current environment is limited to indoor scenes with static terrains, which constrains the diversity of scenarios. Training a locomotion policy from scratch for each task takes approximately three days, posing a significant time burden. Additionally, meshes reconstructed using only RGB input often contain structural imperfections and require manual post-processing, as they lack the fidelity that could be provided by additional sensing modalities such as depth cameras or LiDAR.

6.2. Conclusion

We introduce VR-Robo, a real-to-sim-to-real framework designed to train visual locomotion policies within a photorealistic and physically interactive simulation environment. Extensive experimental results demonstrate that our agents successfully learn robust and effective policies for challenging high-level tasks and can be zero-shot deployed to diverse real-world scenarios. In future work, we aim to extend our framework to larger-scale and more complex environments, incorporating generative models into scene reconstruction for more generalizable policy learning.

References

- [1] Gwangbin Bae and Andrew J Davison. Rethinking inductive biases for surface normal estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9535–9545, 2024. 3
- [2] Arunkumar Byravan, Jan Humplik, Leonard Hasenclever, Arthur Brussee, Francesco Nori, Tuomas Haarnoja, Ben Moran, Steven Bohez, Fereshteh Sadeghi, Bojan Vujatovic, et al. Nerf2real: Sim2real transfer of vision-guided bipedal motion skills using neural radiance fields. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9362–9369. IEEE, 2023. 1, 2
- [3] Edwin Earl Catmull. *A subdivision algorithm for computer display of curved surfaces*. The University of Utah, 1974. 5
- [4] Danpeng Chen, Hai Li, Weicai Ye, Yifan Wang, Weijian Xie, Shangjin Zhai, Nan Wang, Haomin Liu, Hujun Bao, and Guofeng Zhang. Pgsr: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction. *arXiv preprint arXiv:2406.06521*, 2024. 3
- [5] Zoey Chen, Zhao Mandi, Homanga Bharadhwaj, Mohit Sharma, Shuran Song, Abhishek Gupta, and Vikash Kumar. Semantically controllable augmentations for generalizable robot learning. *The International Journal of Robotics Research*, page 02783649241273686, 2024. 2
- [6] Xuxin Cheng, Kexin Shi, Ananye Agarwal, and Deepak Pathak. Extreme parkour with legged robots. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11443–11450. IEEE, 2024. 1, 5
- [7] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*, pages 303–312, 1996. 4
- [8] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 6, 7
- [9] Qiancheng Fu, Qingshan Xu, Yew Soon Ong, and Wenbing Tao. Geo-neus: Geometry-consistent neural implicit surfaces learning for multi-view reconstruction. *Advances in Neural Information Processing Systems*, 35:3403–3416, 2022. 3
- [10] Silvano Galliani, Katrin Lasinger, and Konrad Schindler. Massively parallel multiview stereopsis by surface normal diffusion. In *Proceedings of the IEEE international conference on computer vision*, pages 873–881, 2015. 3
- [11] Antoine Guédon and Vincent Lepetit. Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5354–5363, 2024. 7
- [12] David Hoeller, Nikita Rudin, Dhionis Sako, and Marco Hutter. Anymal parkour: Learning agile navigation for quadrupedal robots. *Science Robotics*, 9(88):eadi7566, 2024. 1, 5
- [13] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019. 7
- [14] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. *arXiv preprint arXiv:2403.17888*, 2024. 3
- [15] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaa5872, 2019. 2
- [16] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 1, 2, 3, 5
- [17] Xiaofan Li, Yifu Zhang, and Xiaoqing Ye. Drivingdiffusion. Layout-guided multi-view driving scene video generation with latent diffusion model. *arXiv preprint arXiv:2310.07771*, 2(3), 2023. 2
- [18] Xinhai Li, Jialin Li, Ziheng Zhang, Rui Zhang, Fan Jia, Tiancai Wang, Haoqiang Fan, Kuo-Kun Tseng, and Ruiping Wang. Robogsim: A real2sim2real robotic gaussian splatting simulator. *arXiv preprint arXiv:2411.11839*, 2024. 1, 2
- [19] Haozhe Lou, Yurong Liu, Yike Pan, Yiran Geng, Jianteng Chen, Wenlong Ma, Chenglong Li, Lin Wang, Hengzhen Feng, Lu Shi, et al. Robo-gs: A physics consistent spatial-temporal model for robotic arm with hybrid representation. *arXiv preprint arXiv:2408.14873*, 2024. 1, 2
- [20] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science robotics*, 7(66):eabm6074, 2022. 7
- [21] Viktor Makovychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021. 1
- [22] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1, 2
- [23] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 3803–3810. IEEE, 2018. 2
- [24] Alex Quach, Makram Chahine, Alexander Amini, Ramin Hasani, and Daniela Rus. Gaussian splatting to real world flight navigation transfer with liquid networks. *arXiv preprint arXiv:2406.15149*, 2024. 2
- [25] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4104–4113, 2016. 3
- [26] Andrew Szot, Alexander Clegg, Eric Undersander, Erik Wi-jmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Singh Chaplot, Oleksandr Maksymets, et al. Habitat 2.0: Training home assistants to rearrange their

- habitat. *Advances in neural information processing systems*, 34:251–266, 2021. [1](#)
- [27] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. *arXiv preprint arXiv:1804.10332*, 2018. [2](#)
- [28] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, et al. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024. [1](#)
- [29] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017. [2](#)
- [30] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012. [1](#)
- [31] Marcel Torme, Anthony Simeonov, Zechu Li, April Chan, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. *arXiv preprint arXiv:2403.03949*, 2024. [2](#)
- [32] Matias Turkulainen, Xuqian Ren, Iaroslav Melekhov, Otto Seiskari, Esa Rahtu, and Juho Kannala. Dn-splatter: Depth and normal priors for gaussian splatting and meshing. *arXiv preprint arXiv:2403.17822*, 2024. [3](#)
- [33] Lirui Wang, Yiyang Ling, Zhecheng Yuan, Mohit Shridhar, Chen Bao, Yuzhe Qin, Bailin Wang, Huazhe Xu, and Xiaolong Wang. Gensim: Generating robotic simulation tasks via large language models. *arXiv preprint arXiv:2310.01361*, 2023. [2](#)
- [34] Jinze Wu, Guiyang Xin, Chenkun Qi, and Yufei Xue. Learning robust and agile legged locomotion using adversarial motion priors. *IEEE Robotics and Automation Letters*, 2023. [1](#), [5](#)
- [35] Yuxuan Wu, Lei Pan, Wenhua Wu, Guangming Wang, Yanzi Miao, and Hesheng Wang. RL-gsbridge: 3d gaussian splatting based real2sim2real method for robotic manipulation learning. *arXiv preprint arXiv:2409.20291*, 2024. [1](#), [2](#)
- [36] Ziyang Xie, Zhizheng Liu, Zhenghao Peng, Wayne Wu, and Bolei Zhou. Vid2sim: Realistic and interactive simulation from video for urban navigation. *arXiv preprint arXiv:2501.06693*, 2025. [1](#), [2](#)
- [37] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xianggang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *arXiv preprint arXiv:2406.09414*, 2024. [2](#), [3](#)
- [38] Alan Yu, Ge Yang, Ran Choi, Yajvan Ravan, John Leonard, and Phillip Isola. Learning visual parkour from generated images. In *8th Annual Conference on Robot Learning*, 2024. [1](#), [2](#), [7](#)
- [39] Shaoting Zhu, Runhan Huang, Linzhan Mou, and Hang Zhao. Robust robot walker: Learning agile locomotion over tiny traps. *arXiv preprint arXiv:2409.07409*, 2024. [5](#)
- [40] Shaoting Zhu, Derun Li, Linzhan Mou, Yong Liu, Ningyi Xu, and Hang Zhao. Saro: Space-aware robot system for terrain crossing via vision-language model. *arXiv preprint arXiv:2407.16412*, 2024. [7](#), [8](#)
- [41] Ziwen Zhuang, Zipeng Fu, Jianren Wang, Christopher Atkeson, Soeren Schwertfeger, Chelsea Finn, and Hang Zhao. Robot parkour learning. *arXiv preprint arXiv:2309.05665*, 2023. [1](#), [5](#)
- [42] Ziwen Zhuang, Shenzhe Yao, and Hang Zhao. Humanoid parkour learning. *arXiv preprint arXiv:2406.10759*, 2024. [1](#)